

Key questions:

Does training still converge with large delays?
Does it converge to the same solution as sync SGD?
How does delay interact with learning rate?

Project Roadmap: Async SGD — Delay, Momentum & Regularization

The Common Thread

Your project tells one coherent story:

"Asynchronous SGD introduces gradient staleness — we systematically study how this staleness affects optimization, how momentum interacts with it, and whether it acts as an implicit regularizer."

Each step builds on the previous one. Delay is the **independent variable** throughout the entire project.

Step 1 — Setup & Baseline

Goal: Establish a clean, reproducible experimental framework that all later experiments build on.

Choices to make & justify:

- What model? (recommendation: a small ResNet or MLP — simple enough to train fast, complex enough to be interesting)
- What dataset? (recommendation: CIFAR-10 or MNIST — well understood, fast to train)
- What is your "synchronous SGD" baseline? Fix its hyperparameters (lr, batch size, epochs) carefully — this is your reference point for everything

Key questions to answer:

- Does your sync SGD baseline reach a reasonable accuracy? (validates your setup)
- How do you simulate delay in PyTorch? (buffer gradients from τ steps ago — document this clearly)
- Is your simulation of async SGD faithful to real distributed async training?

Step 2 — Effect of Delay on Convergence

Goal: Understand the fundamental impact of staleness alone, before adding any complexity.

What you do:

- Train with fixed delays $\tau = 0, 2, 5, 10, 20$ (and maybe 50 if stable)
- No momentum for now — isolate the variable
- Plot training loss curves and final test accuracy for each τ

Key questions to answer:

- Is there a **critical delay threshold** beyond which convergence degrades significantly?
- Does larger delay just slow convergence, or does it converge to a **worse solution**?
- Does the **learning rate** need to be adapted as delay grows? (theory suggests scaling η by $1/\tau$ — does this hold empirically?)
- Is the degradation **gradual or sudden** as τ increases?

This step is the foundation. It tells you what "damage" delay does, so the next steps can ask whether momentum makes it better/worse and whether the damage has a silver lining.

Step 3 — Interaction Between Delay and Momentum

Goal: Understand whether momentum helps or hurts under staleness, and why.

What you do:

- Repeat Step 2 experiments but now with momentum ($\beta = 0.9$ typically)
- Compare: async SGD without momentum vs async SGD with momentum, across the same delay values τ
- Try different momentum values ($\beta = 0, 0.5, 0.9$) at a fixed moderate delay (e.g. $\tau = 10$)

Key questions to answer:

- At **small delays**, does momentum still help as expected?
- At **large delays**, does momentum amplify the staleness problem (gradients from the past get accumulated and push in wrong directions)?
- Is there an **optimal momentum value** that depends on the delay?
- Does the interaction change the critical threshold you found in Step 2?

The expected finding (which you verify or challenge): momentum helps at low delay but becomes harmful at high delay because it amplifies stale gradient contributions. This is a clean, narratable result.

Step 4 — Regularization Effect

Goal: Determine whether gradient staleness acts as implicit regularization, analogous to dropout.

What you do:

- Compare the **generalization gap** (train accuracy – test accuracy) across delay values
- Measure the **sharpness of minima** found: compute or approximate the largest eigenvalue of the Hessian at convergence (libraries like PyHessian make this feasible)
- Compare against an explicit regularization baseline: same model trained with dropout

Key questions to answer:

- Does higher delay lead to **flatter minima** (lower sharpness)?
- Does higher delay improve **test accuracy** even if it slightly hurts train accuracy?
- Is the regularization effect of delay **comparable in magnitude** to dropout?
- Is there a **sweet spot**: a delay large enough to regularize but small enough not to break convergence? (this would be your most interesting finding)

This step reframes delay from "a problem to minimize" to "a potentially useful property" — which is a compelling scientific narrative.

Step 5 — Synthesis & Writing

Goal: Tie all results into a coherent 3-page story.

Key questions to answer:

- What is the **single most important finding** of your project? (lead with this)
- Do your results **agree or disagree** with existing literature? (Recht et al. Hogwild!, Mitliagkas et al. 2016 on momentum and async)
- What would be the **practical recommendation** for someone training a model asynchronously?

Suggested paper structure:

1. Introduction — motivate async SGD, state your three research questions
2. Methods — describe your delay simulation, model, dataset, metrics
3. Experiments — one section per step above, each with a clear takeaway
4. Conclusion — the unified answer to: "*should you use async SGD, with what delay, and with momentum?*"

Summary Table

Step	Core Question	Key Variable
1	Does our setup work?	—
2	How does delay hurt convergence?	τ (delay)
3	Does momentum help or hurt under delay?	$\tau \times \beta$ (momentum)
4	Does delay regularize like dropout?	τ vs generalization
5	What's the unified story?	—

The strength of this structure is that **each step answers a question that motivates the next one**. Step 2 shows delay has a cost → Step 3 asks if momentum compensates → Step 4 asks if the cost actually has a benefit. That's a clean scientific arc for a 3-page paper.

Want help thinking about the PyTorch implementation of the delay simulation?